

Here is the complete, end-to-end documentation of all 21 nodes in your Mycroft Regulatory Intelligence n8n workflow, summarizing their functions, execution behaviors, and critical architectural vulnerabilities identified during our inspection:

Phase 1: Ingestion & Trigger (Nodes 1–7)

Node 1: Schedule Trigger (**scheduleTrigger**)

- **Function:** Fires automatically on a daily schedule configured for 10:00 AM UTC.
- **Payload:** Passes execution timestamp metadata (**scraped_at**, formatting strings) downstream.
- **Status:**  Operational.

Nodes 2–6: RSS Feed Readers (**rssFeedRead**)

- **Feeds:** SEC Press Releases, CFTC Federal Register, Securities Federal Register, FINRA Google News, and Investment Advisor Google News.
- **Function:** Fetches raw RSS XML items from the respective regulatory endpoints.
- **Status:**  Vulnerable (Google News RSS items return wrapped redirect URLs instead of canonical target links).

Node 7: Merge (**merge**)

- **Function:** Consolidates incoming items from all 5 RSS feeds into a single unified array.
- **Status:**  Operational.

Phase 2: Normalization & Filtering (Nodes 8–9)

Node 8: Normalize Data (**code**)

- **Function:** Standardizes article JSON fields, attempts to unwrap Google News redirect links, tags the **source_feed**, and strips duplicates using a **lower(title) + date** hash.
- **Flaw:** Lacks specific Federal Register classification logic, causing non-financial rules (e.g., environmental or wildlife notices) to be mislabeled as "**Federal Register - Securities**".

Node 9: Filter Valid Content (**if**)

- **Function:** Evaluates whether incoming articles contain non-empty title and content fields.
- **Flaw:** Drops valid SEC press releases that ship with an empty **content** body string (**content: ""**), causing significant data loss.

Phase 3: Scoring, Preparation & Database Loading (Nodes 10–13)

Node 10: Keyword Analysis & Urgency Scoring (**code**)

- **Function:** Scans article titles and bodies against 6 keyword categories (**enforcement, compliance, crypto, securities, derivatives, fraud**) and assigns numeric urgency scores (\$5\$ to \$10\$) mapping to impact tiers (**Critical, High, Medium**).
- **Flaw:** Hardcoded baseline score of **5** inflates non-financial noise to "Medium" impact.

Node 11: Prepare Data (**code**)

- **Function:** Formats text fields specifically for raw SQL string interpolation by manually escaping single quotes (' \$\\rightarrow\$ ") and stringifying JSON metadata.
- **Flaw:** Bypasses native SQL parameterized queries, creating execution fragility.

Node 12: Insert Data into DB (**postgres**)

- **Function:** Executes a raw string-interpolated `INSERT INTO regulatory_feeds ... ON CONFLICT (title, DATE(published)) DO NOTHING RETURNING *;`
- **Flaw:** Relies on string interpolation instead of safe driver bindings; any unescaped special character crashes the entire batch insert. Additionally, local environments require proper local database configuration (e.g., port `5431` on PostgreSQL.app).

Node 13: Code in JavaScript (**code**)

- **Function:** Evaluates database insert returns, filters successfully written records (`id` exists), and logs batch insertion counts.
- **Flaw:** Relies on volatile in-memory console logs rather than a persistent metrics collector.

Phase 4: Routing & Reporting (Nodes 14–16)

Node 14: If2 (**if**)

- **Function:** Splits the execution path depending on whether new items exist (`newItems.length > 0`).
 - **True Branch:** Proceeds to HTML report generation and alert processing.
 - **False Branch:** Routes to an "All Clear" email notification.
- **Status:**  Operational.

Node 15: Generate HTML Report (**code**)

- **Function:** Aggregates statistics across the batch (total updates, critical items, enforcement counts, category distribution, source breakdowns) and compiles a styled HTML digest.
- **Flaw:** Threshold mismatch (`urgency_score > 7` in the report summary vs. `>6` in downstream alert filters) creates a reporting discrepancy.

Node 16: Read/Write Files from Disk (**readWriteFile**)

- **Function:** Saves the generated HTML report locally.
- **Flaw:** Hardcoded to a Linux path (`/home/therrrshan/mycroft`) which triggers an `ENOENT` error when executed on a local macOS environment (`/Users/pranavijs/`).

Phase 5: Alerting & Email Dispatch (Nodes 17–21)

Node 17: High Priority Filter (**code** / **if**)

- **Function:** Filters articles matching alert criteria (`urgency_score > 6` or high impact) for immediate notification.

Node 18: Generate Email (**code**)

- **Function:** Constructs the HTML body for high-priority regulatory alerts or the alternative "All Clear" status email.

Node 19: If (**if**)

- **Function:** Validates whether any high-priority alerts require immediate email dispatch.

Node 20 & 21: Send Email / Send Email Alert (email)

- **Function:** Dispatches the final HTML alert email or status update via SMTP and updates tracking flags (`email_sent = TRUE`) in PostgreSQL.